A4: Style Transfer with Deep Neural Networks

CSCI 611 – Spring 2026

Arushi Tewari

1. Introduction

This assignment implements neural style transfer based on the paper Image Style Transfer Using Convolutional Neural Networks by Gatys et al. (2016). The method uses a pre-trained VGG19 convolutional neural network to extract content and style representations from images, and iteratively updates a target image to minimize both content and style losses simultaneously. The result is a new image that preserves the structural content of one image while adopting the artistic style of another.

For this implementation, a photograph of the New York City skyline was used as the content image, and a floral still-life painting was used as the style image.

2. Implementation

2.1 Model Architecture

The implementation uses the features portion of VGG19, a deep convolutional neural network pre-trained on ImageNet. The classifier layers are discarded since only intermediate feature representations are needed. All VGG19 parameters are frozen — only the target image pixels are updated during optimization.

2.2 Layer Mapping (TODO 1)

The VGG19 layers were mapped to their paper names as follows:

```
layers = {'0':  'conv1_1',   # style
          '5':  'conv2_1',   # style
          '10': 'conv3_1',   # style
          '19': 'conv4_1',   # style
          '21': 'conv4_2',   # content
          '28': 'conv5_1'}   # style
```

Layer conv4_2 is used for content representation as specified in the paper, as it captures high-level structural information. Layers conv1_1 through conv5_1 are used for style representation, capturing textures and patterns at multiple scales.

2.3 Gram Matrix (TODO 2)

The Gram matrix captures style by measuring correlations between feature maps. It is computed by reshaping the feature tensor and multiplying it by its transpose:

```
def gram_matrix(tensor):
    batch_size, d, h, w = tensor.size()
    tensor = tensor.view(batch_size * d, h * w)
    gram = torch.mm(tensor, tensor.t())
    return gram
```

2.4 Loss Functions (TODOs 3 & 4)

Content Loss measures the mean squared difference between target and content features at conv4_2:

```
content_loss = torch.mean((target_features['conv4_2'] -
content_features['conv4_2'])**2)
```

Style Loss is computed across all style layers by comparing gram matrices, weighted by layer importance and normalized by layer size:

```
target_gram = gram_matrix(target_feature)
style_gram = style_grams[layer]
layer_style_loss = style_weights[layer] * torch.mean((target_gram -
style_gram)**2)
style_loss += layer_style_loss / (d * h * w)
```

Total Loss combines both losses weighted by alpha (content_weight) and beta (style_weight):

```
total_loss = content_weight * content_loss + style_weight * style_loss
```

3. Baseline Experiment (Experiment 1)

The baseline experiment used the following default hyperparameters as suggested in the notebook:

- content_weight (alpha) = 1
- style_weight (beta) = 1e6
- steps = 2000
- learning rate = 0.003 (Adam optimizer)
- style_weights: conv1_1=1.0, conv2_1=0.8, conv3_1=0.5, conv4_1=0.3, conv5_1=0.1

The input images and final stylized result are shown below:

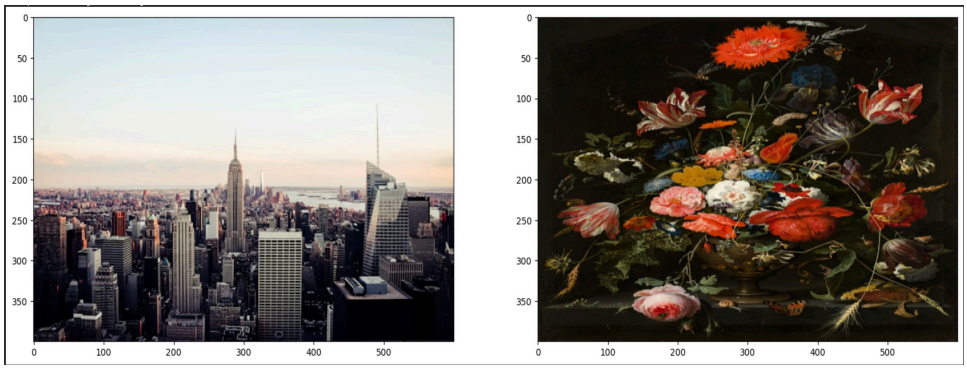

Figure 1: Content image (NYC skyline, left) and style image (floral still-life painting, right)

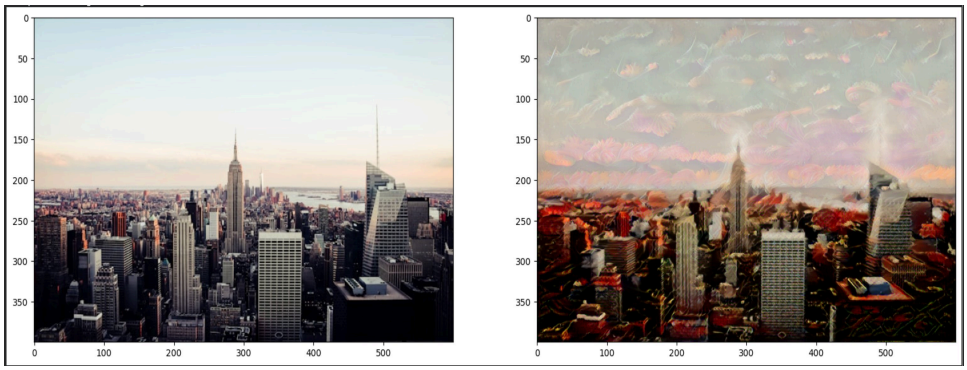

Figure 2: Experiment 1 result — baseline settings (style_weight=1e6). The city structure is preserved with subtle painterly textures.

4. Hyperparameter Experiments (Part 2)

Four experiments were conducted by varying the style weight and layer weights to observe their effect on the stylization outcome.

Experiment	style_weight	Layer Weights	Description
1 (Baseline)	1e6	Early layers emphasized (conv1_1=1.0 → conv5_1=0.1)	Balanced style and content
2	1e8	Early layers emphasized (conv1_1=1.0 → conv5_1=0.1)	Very high style dominance
3	1e4	Early layers emphasized (conv1_1=1.0 → conv5_1=0.1)	Low style, content preserved
4	1e6	Later layers emphasized (conv1_1=0.1 → conv5_1=1.0)	Fine-grained style features

4.1 Experiment 2: High Style Weight (1e8)

Increasing the style weight to 1e8 dramatically increases style influence relative to content. The painting's colors and textures overwhelm the city structure, with large floral shapes from the style image clearly visible across the entire image.

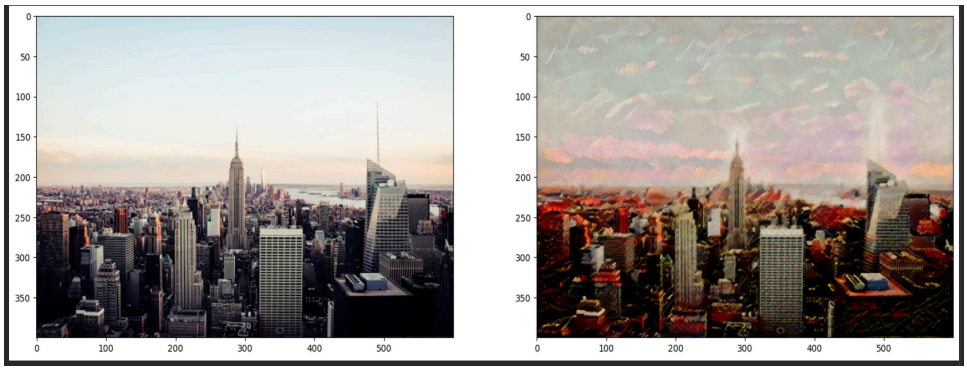

Figure 3: Experiment 2 (style_weight=1e8) — aggressive stylization, city structure heavily distorted by painting textures.

4.2 Experiment 3: Low Style Weight (1e4)

Lowering the style weight to 1e4 reduces style influence significantly. The target image retains most of the original city photograph's appearance, with only subtle color shifts and texture changes visible. This demonstrates that a very low style weight prioritizes content fidelity over artistic style.

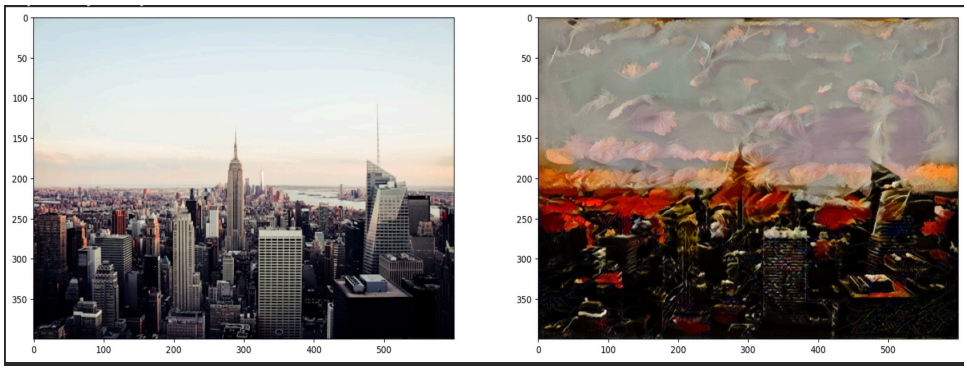

Figure 4: Experiment 3 (style_weight=1e4) — subtle stylization, city structure well preserved with mild color influence.

4.3 Experiment 4: Later Layer Emphasis

Reversing the layer weights to emphasize later convolutional layers (conv4_1 and conv5_1) over earlier ones produces finer, more detailed style patterns. Earlier layers capture large-scale textures like brushstroke patterns; later layers capture more subtle, fine-grained features. The result shows more delicate style transfer compared to the baseline.

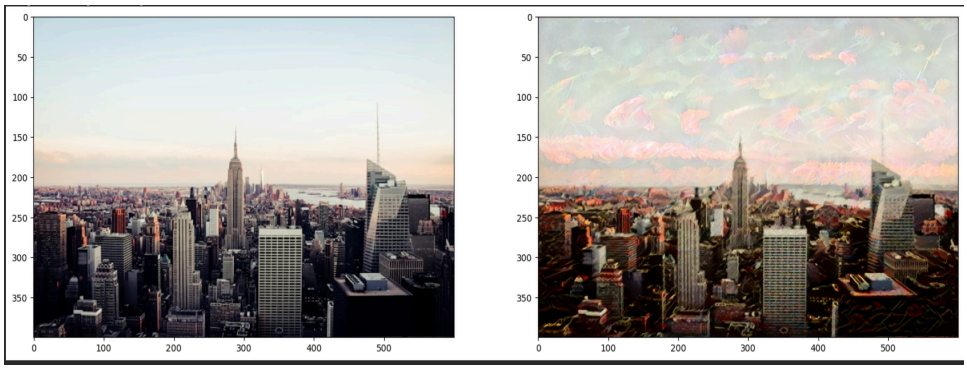

Figure 5: Experiment 4 (later layers emphasized) — finer style details, more delicate textures compared to baseline.

5. Analysis and Findings

5.1 Effect of Style Weight (alpha/beta ratio)

The style_weight hyperparameter has the most dramatic effect on the output. As shown across Experiments 1-3:

- style_weight=1e4: Minimal stylization — the image looks nearly identical to the original photograph
- style_weight=1e6: Balanced result — clear painterly style while preserving city structure
- style_weight=1e8: Over-stylized — the painting dominates and the content is largely lost

This confirms the paper's finding that the alpha/beta ratio is the primary control for balancing content and style in the output image.

5.2 Effect of Layer Weights

Emphasizing earlier layers (conv1_1, conv2_1) results in larger, coarser style artifacts — the broad color palette and large brushstroke patterns from the painting. Emphasizing later layers (conv4_1, conv5_1) produces finer, more subtle texture transfer. This is because earlier layers in VGG19 respond to low-level features like edges and colors, while later layers capture more abstract, high-level patterns.

5.3 Loss Convergence

The total loss decreased consistently across all 2000 steps in every experiment. In Experiment 1, the loss dropped from approximately 45 million at step 400 to 11 million at step 2000, demonstrating steady convergence. Higher style weights produced larger absolute loss values (Experiment 2 showed losses in the hundreds of millions) but still converged smoothly.

5.4 Observations

- The sky region of the content image was most affected by style transfer in all experiments, likely because it has less structural detail for the content loss to preserve
- The building structures in the lower half of the image were better preserved across all experiments, demonstrating the content loss effectively anchoring structural elements
- The GPU (T4 on Google Colab) completed 2000 steps in approximately 5 minutes — significantly faster than CPU training

6. Conclusion

This assignment successfully implemented neural style transfer following the Gatys et al. (2016) paper. The four TODOs — layer mapping, gram matrix computation, content loss, and style/total loss — were completed and verified through successful stylization results. Hyperparameter experiments demonstrated that the style_weight is the most impactful parameter, controlling the balance between content preservation and artistic stylization. Layer weight experiments showed that earlier layers capture coarser style features while later layers capture finer details, allowing multi-scale control over the style transfer output.